

BE-1 — Start Here

Physics & Simulation. Layers 0–1. PS components A, B, C.

The full picture is in `work-breakdown.md`. This is what to do **today**, in order. Do not read the 29-page document first — do task 1, then read.

The order matters, and it is not the obvious one

The obvious order is: build the engine model, get it right, then connect it.

Do the opposite. Publish a socket that emits schema-valid frames *before* the physics is real, verify the dashboard consumes it, and only then put a mean-value engine model behind it.

Why: integration is where hackathon teams die on day five. If the wire works on day one, everything after is improving a connected system instead of praying two halves meet. And you get a visible win in the first hour — the dashboard badge flips from `SIMULATED · NO BACKEND` to `LIVE`, in front of the whole team.

Task 1 — Make the dashboard say LIVE (target: first 90 minutes)

1.1 Get the frontend running so you can see your own output

```
git clone https://github.com/dev591/sih2026.git
cd sih2026/sih-project/frontend
npm install
npm run dev      # http://localhost:5173
```

Top right will say **SIMULATED · NO BACKEND** in amber. Your job is to make it say **LIVE** in green.

1.2 Read exactly one document

`docs/spec/telemetry-schema.md` — specifically the `pramana.health.v1` block. That is the contract. Everything else can wait.

1.3 Publish a stub

```
cd ../backend
python3 -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn numpy pyyaml
```

Serve `ws://localhost:8000/ws/telemetry`, emitting one JSON envelope per second:

```
{ "slow": {...}, "fast": {...}, "health": {...}, "predicted": {...} }
```

Shapes exactly per the spec. **Constant plausible numbers are fine for now** — the point is the wire, not the values. The frontend validates `health.schema == "pramana.health.v1"` and ignores anything malformed rather than blanking the dashboard, so a half-built frame is safe to send.

1.4 Verify

Reload the dashboard. The badge should go green within a second or two. If it does not, hover it — the tooltip carries the reason.

When the badge is green, task 1 is done. Commit it. That is the single most valuable commit you will make this week, because it retires the integration risk on day one.

The frontend retries with backoff in the background, so you can start and stop your server freely — the dashboard upgrades to live whenever you come up, with no reload.

Task 2 — The mean-value engine model (rest of day 1 into day 2)

Now replace the constants with physics. Five state groups.

```
backend/twin/atmosphere.py    ISA + offset – 3 lines, do it first
backend/twin/profiles.py      load config/engine_vrde_180.yaml
backend/twin/mvem.py          the five states
```

1. Intake manifold — filling and emptying

$$dp_{im}/dt = (R \cdot T_{im} / V_{im}) \cdot (\dot{m}_c - \dot{m}_a)$$

Turbocharger degradation shows up here first, as an inability to hold boost.

2. Cylinder induction — speed–density

$$\dot{m}_a = \eta_v(p_{im}, N) \cdot p_{im} \cdot V_d \cdot N / (R \cdot T_{im} \cdot 120)$$

The 120 is 2 crank revolutions per cycle $\times$ 60 s/min. η_v is the channel through which ring wear and valve recession enter the model.

3. Crankshaft

$$J \cdot d\omega/dt = T_{ind} - T_{fric} - T_{pump} - T_{load}$$
$$T_{ind} = \eta_i \cdot \dot{m}_f \cdot Q_{LHV} / \omega$$

Accumulate indicated torque PER CYLINDER. This matters: suppress one cylinder's contribution and you get a physically correct misfire signature, including the 0.5-order crank ripple, instead of an imposed one.

4. Cylinder head thermal — one state PER CYLINDER

$$m \cdot c_p \cdot dT_{cht,i}/dt = \dot{Q}_{gas,i} - h_{air} \cdot A_{fin} \cdot (T_{cht,i} - T_{cool})$$

Four independent thermal states are what make per-cylinder localisation physically meaningful. **Do not lump them.**

5. Turbocharger shaft

$$J_{tc} \cdot \omega_{tc} \cdot d\omega_{tc}/dt = \eta_m \cdot P_{turb} - P_{comp}$$
$$P_{comp} = (\dot{m}_a \cdot c_p \cdot T_1 / \eta_c) \cdot [(p_2/p_1)^{(\gamma-1)/\gamma} - 1]$$

Scale η_c down and you get compressor degradation that is **invisible at sea level and mission-limiting above critical altitude** — 11 000 ft for the VRDE.

Read the frontend's stand-in first

`frontend/src/mock/missionGenerator.ts` is a reduced version of exactly this, and it currently drives the whole dashboard. **Read it as a shape reference — not to copy the physics, which is deliberately simplified — then delete it once yours is live.**

It obeys the two rules yours must:

1. **Faults are PARAMETER PERTURBATIONS, never spikes pasted on a signal.** Degrade an injector discharge coefficient and let EGT, fuel flow, λ and the crank ripple move on their own. You never hand-author what a fault looks like.
 2. **A SENSOR fault perturbs the MEASUREMENT ONLY, never the engine state.** This asymmetry is the entire sensor-vs-engine discriminator. If it is not true in your generator, the headline demo is a lie.
-

Task 3 — Residuals ρ_1 – ρ_{11} (day 2)

`backend/parity/residuals.py`, per `docs/spec/residual-spec.md`.

Two traps an expert evaluator will check:

1. **With n estimates of one quantity, only $n-1$ residuals are independent.** The third pairwise difference is the sum of the first two. A system claiming six residuals from four estimates is double-counting.
2. **Path 4 does not exist on an unthrottled FADEC aero-diesel.** Read `parity_paths` from the engine YAML.
Return `null` for ρ_3 on the VRDE. **Never fabricate it.**

ρ_6 – ρ_9 are deviation from the **conditional mean across cylinders**, not from an absolute value. That is what makes them operating-point invariant — and it means they sum to zero, which matters for the novelty projection (ask BE-2).

Definition of done, per day

DAY	DONE MEANS
1	Dashboard badge is green . MVEM states 1–3 running with ISA.
2	Thermal + turbo states. Six faults. Healthy dataset generated and handed to BE-2. This is the Day-2 tag.
3	All ten faults. Damage integrator with ground-truth RUL labels.
4	Mission forward-propagation, Monte Carlo over degradation posterior.
5	CAN on <code>vcan0</code> , <code>candump</code> visible in the demo.

What you must not do

- **Do not** build a crank-angle-resolved model. Wrong fidelity — no flight sensor observes in-cylinder pressure, and it will not run fast enough for the Monte Carlo mission layer.
 - **Do not** inject faults as signal spikes. The entire value is that they propagate through physics.
 - **Do not** hardcode an engine constant. Everything engine-specific comes from `config/engine_*.yaml`. "A new engine is a config change" is one of our five differentiating claims and it has to stay true in your code as it now is in the frontend's.
 - **Do not** change a schema field name without telling BE-2 and the frontend *before* you push. Adding a field is free; renaming one on day 3 costs more than the feature was worth.
 - **Do not** tune the model to make the demo look good. If the physics says something, the demo shows it.
-

Where to ask

- Contract questions → `docs/spec/telemetry-schema.md` , then the frontend lead
- Residual definitions → `docs/spec/residual-spec.md`
- Why any of this → `docs/SOLUTION-ANALYSIS.md` §2